

Binary Classifications Report

By Jay Vardhan Ghildiyal, a1953779

Task 1 : Data Exploration and Analysis

Data Inspection

we start by loading our dataset and analyzing the data inside



Python

Cell 1

```
1 print(type(data))
2 print(list(data.keys()))
```

```
<class 'numpy.lib.npyio.NpzFile'>
['train_data', 'train_targets', 'test_data', 'test_targets']
```

the dataset is a `numpy.lib.npyio.NpzFile` file, which is a NumPy zip file object. it's basically a container that holds multiple arrays inside it (like a dictionary).

the array of strings above are the labels of our datasets present inside that container.



Python

Cell 2

```
1 # create different arrays for each key
2 X_train = data["train_data"]
3 y_train = data["train_targets"]
4 X_test  = data["test_data"]
5 y_test  = data["test_targets"]
6
7 print("X_train shape:", X_train.shape)
8 print("y_train shape:", y_train.shape)
9 print("X_test shape: ", X_test.shape)
10 print("y_test shape: ", y_test.shape)
```

```
X_train shape: (100, 28, 28)
y_train shape: (100,)
X_test shape: (1000, 28, 28)
y_test shape: (1000,)
```

we have 100 samples for training and a 1000 samples for testing. each sample has a 28x28 grayscale image (784 features or pixels) with it.

the image is present in the form of a 28x28 matrix. each singular value in that matrix represents a pixel.

each pixel has a numerical value that representing it's brightness, ranging from 0 to 255 :

- 0 = black
- 255 = white
- anything in between 0 to 255 = a shade of grey

we can also observe that the distribution of data (atleast in the test set) is skewed towards the **digit 7**

```
training labels: [1 7]
```

```
class counts in training set : ``digit 1 : 50 samples  digit 7 : 50 samples`
```

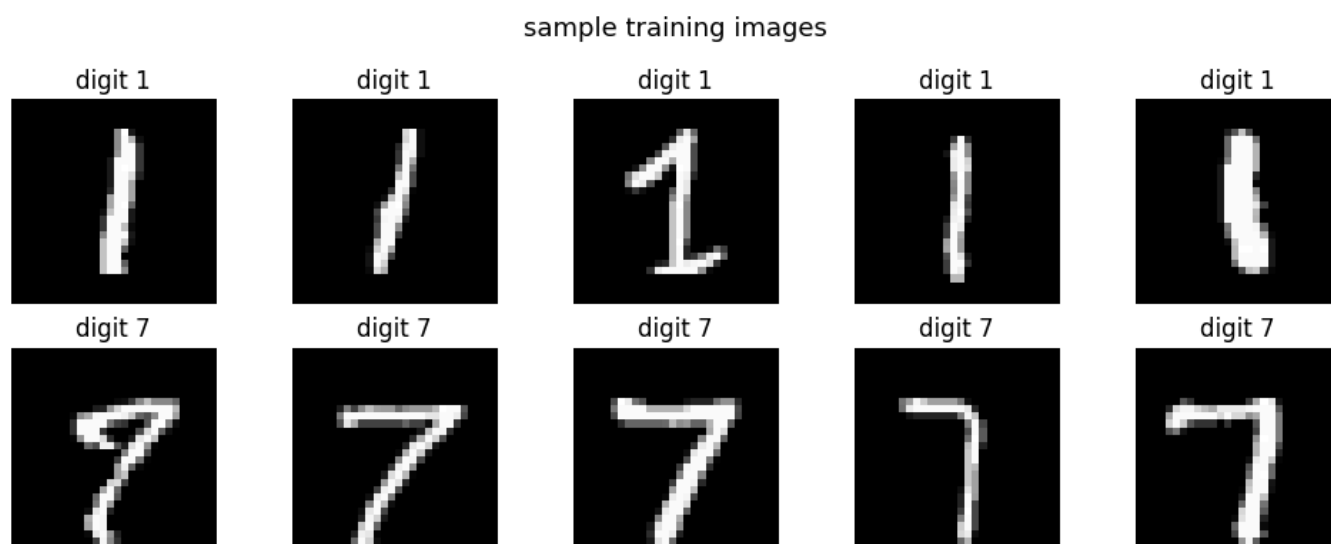
```
Class counts in test set :
```

```
digit 1 : 100 samples
```

```
digit 7 : 900 samples
```

this means that if our model learns to predict the **digit 7** for the most part, it would give a false sense of accuracy. this warrants using metrics like precision, recall and F1 score.

using the 2D matrices, i've constructed the first five digits for visualization.



this image was created using the `matplotlib` library. as handwriting goes, most digits look similar, even if the style slightly varies.

Data Pre-Processing

since linear classifiers expect to use a 1D vector for features, we need to flatten our 28x28 2D matrix into a 1D array.



Python

Cell 5

```
1 # flatenning the images from 28x28 to 784
2 X_train_flat = X_train.reshape(100, -1)
3 X_test_flat  = X_test.reshape(1000, -1)
```

```
X_train_flat shape : (100, 784)
```

```
X_test_flat shape : (1000, 784)
```

we also normalize the pixel brightness values from 0-255 to 0-1. since we don't have varying ranges in our training data, we don't really need to do this. but it's considered good practice to do it anyways.



Python

Cell 5

```
1 # normalizing the pixel values / features
2 X_train_flat = X_train_flat / 255.0
3 X_test_flat  = X_test_flat  / 255.0
```

we normalize our data by dividing it by 255 (highest pixel value). it can have multiple advantages :

- numerical stability : models internally do lots of multiplications and additions. large values like 255 can cause the numbers to get very large very fast, leading to unstable training or overflow issues
- convergence speed : logistic regression uses gradient descent under the hood. Smaller, consistent values help it converge faster and more reliably

Task 2 : Build a Classification System